

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.3 Message integrity & end point authentication

8.4 Securing e-mail

8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and IDS

11/25/2013

Network Security (SSL) 8-1

What is network security?

Confidentiality: only sender, intended receiver(s) should "understand" message contents

- sender encrypts message
- receiver decrypts message

Authentication: sender, receiver want to confirm identity of each other

Message Integrity: sender, receiver want to ensure that any message that is altered (in transit, or afterwards) will be detected

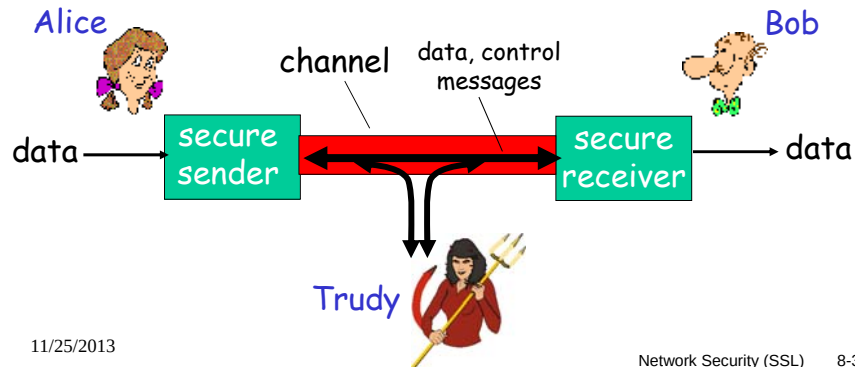
Access control and Availability: services accessible and available to **authorized** users only

11/25/2013

Network Security (SSL) 8-2

Friends and enemies: Alice, Bob, Trudy

- well-known in network security world
- Bob, Alice want to communicate "securely"
- Trudy (intruder) may intercept, delete, insert, modify messages



11/25/2013

Network Security (SSL) 8-3

Who might Bob, Alice be?

- Web browser/server for electronic transactions (e.g., on-line purchases)
- on-line banking client/server
- DNS servers
- routers exchanging routing table updates
- ...

11/25/2013

Network Security (SSL) 8-4

What can a "bad" guy do to attack protocols?

- *eavesdrop*: intercept messages
- actively *insert* messages into connection
- *impersonation*: can fake (spoof) source IP address in packet (or any field in packet)
- *hijack* an ongoing connection by removing sender or receiver, inserting itself in its place, or *inserting itself in the middle*
- *IP prefix hijack*
- *denial of service*: prevent service from being used by others (e.g., by overloading resources)
- ...

11/25/2013

Network Security (SSL) 8-5

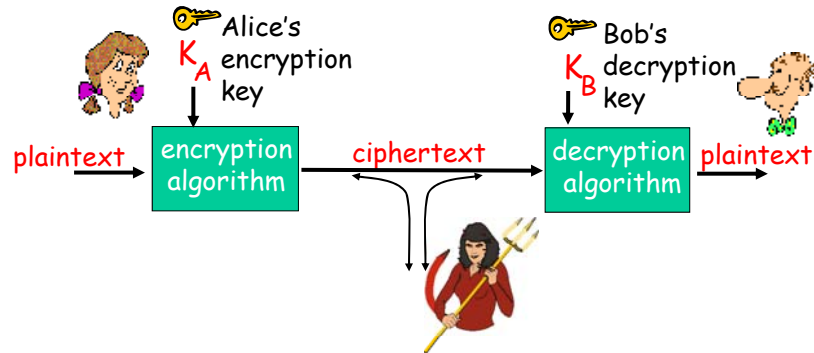
Chapter 8 roadmap

- 8.1 What is network security?
- 8.2 Principles of cryptography
- 8.3 Message integrity & end point authentication
- 8.4 Securing e-mail
- 8.5 Securing TCP connections: SSL
- 8.6 Network layer security: IPsec
- 8.7 Securing wireless LANs
- 8.8 Operational security: firewalls and IDS

11/25/2013

Network Security (SSL) 8-6

The language of cryptography

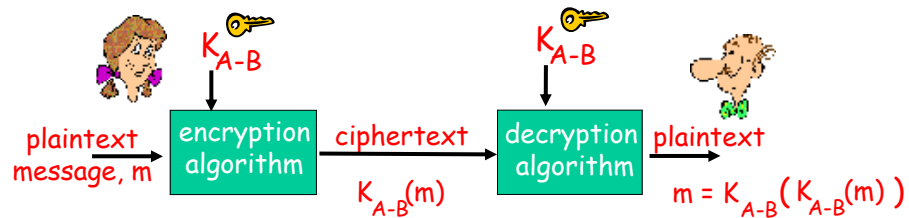


symmetric key crypto: sender, receiver keys *identical*
public-key crypto: encryption key is *public*, decryption key is *private* (also called **asymmetric key** crypto)

11/25/2013

Network Security (SSL) 8-7

Symmetric key cryptography



- **symmetric key crypto:** Bob and Alice share (know) the **same** key
- **standards:** DES, 3DES, AES, RC4, ...

11/25/2013

Network Security (SSL) 8-8

Public Key Cryptography

symmetric key crypto

- requires sender and receiver to share a secret key
- Q: how to agree on key in first place (particularly if never "met")?

public key cryptography

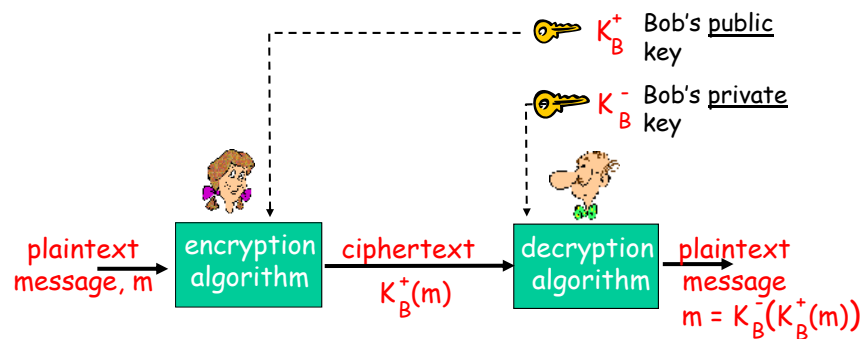
- radically different approach [Diffie-Hellman76, RSA78]
- sender, receiver do *not* share secret key
- a *public* key for encrypting (or verifying)
- a *private* key for decrypting (or signing)



11/25/2013

Network Security (SSL) 8-9

Public key cryptography



11/25/2013

Network Security (SSL) 8-10

Public key encryption algorithms

Requirements:

- ① need $K_B^+(\cdot)$ and $K_B^-(\cdot)$ such that
$$K_B^-(K_B^+(m)) = m$$
- ② given public key K_B^+ , it should be impossible to compute private key K_B^-

RSA: Rivest, Shamir, Adleman algorithm

11/25/2013

Network Security (SSL) 8-11

RSA: Creating public/private key pair

1. choose two large prime numbers p, q
(e.g., 1024 or 2048 bits each)
2. compute $n = pq$, $z = (p-1)(q-1)$
3. choose e (with $e < n$) that has no common factors with z (e, z are “relatively prime”)
4. choose d such that $ed \bmod z = 1$
5. public key is (n, e) , private key is (n, d)

$\underbrace{(n, e)}_{K_B^+}$

$\underbrace{(n, d)}_{K_B^-}$

11/25/2013

Network Security (SSL)

8-12

RSA: encryption, decryption

1. to encrypt message m ($< n$), compute

$$c = m^e \bmod n$$

2. to decrypt received bit pattern, c , compute

$$m = c^d \bmod n$$

magic happens! $m = (\underbrace{m^e \bmod n}_c)^d \bmod n$

11/25/2013

Network Security (SSL)

8-13

RSA toy example:

Bob chooses $p=5$, $q=7$. Then $n=35$, $z=24$

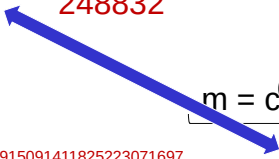
choose $e = 5$ (so e, z relatively prime)

choose $d=29$ (so $ed \bmod 24 = 145 \bmod 24 = 1$)

encrypting 8-bit messages:

encrypt: $\underbrace{\text{bit pattern}}_{00001000} \quad \underbrace{m}_{12} \quad \underbrace{m^e}_{248832} \quad \underbrace{c = m^e \bmod n}_{17}$

decrypt: $\underbrace{c}_{17} \quad \underbrace{c^d}_{481968572106750915091411825223071697} \quad \underbrace{m = c^d \bmod n}_{12}$



11/25/2013

Network Security (SSL)

8-14

Why does RSA work?

□ show that $c^d \bmod n = m$, where $c = m^e \bmod n$

□ from number theory:

for any x and y , $x^y \bmod n = x^{(y \bmod z)} \bmod n$

where $n = pq$ and $z = (p-1)(q-1)$

□ thus,

$$\begin{aligned} c^d \bmod n &= (m^e \bmod n)^d \bmod n \\ &= m^{ed} \bmod n \\ &= m^{(ed \bmod z)} \bmod n \\ &= m^1 \bmod n \\ &= m \end{aligned}$$

11/25/2013

Network Security (SSL)

8-15

Why is RSA secure?

□ suppose you know Bob's public key (n, e) .
How hard is it to determine d ?

□ need to find the factors p and q of n

○ fact: factoring a big number is "hard"

11/25/2013

Network Security (SSL)

8-16

RSA: another important property

From modular arithmetic

$$\begin{aligned}(m^e \bmod n)^d \bmod n &= m^{ed} \bmod n \\ &= m^{de} \bmod n \\ &= (m^d \bmod n)^e \bmod n\end{aligned}$$

$$\underbrace{K_B^-(K_B^+(m))}_{\text{use public key first, followed by private key}} = m = \underbrace{K_B^+(K_B^-(m))}_{\text{use private key first, followed by public key}}$$

use public key
first, followed
by private key

use private key
first, followed
by public key

Result is the same!

11/25/2013

Network Security (SSL) 8-17

Session keys

- RSA requires exponentiation which is computationally intensive
e.g., symmetric key encryption is 100 times faster than RSA

Session key, K_S

- Bob and Alice use RSA to exchange a symmetric key K_S
- Once both have K_S , they use symmetric key crypto for confidential communication

11/25/2013

Network Security (SSL) 8-18

Chapter 8 roadmap

- 8.1 What is network security?
- 8.2 Principles of cryptography
- 8.3 **Message integrity** & end point authentication
- 8.4 Securing e-mail
- 8.5 Securing TCP connections: SSL
- 8.6 Network layer security: IPsec
- 8.7 Securing wireless LANs
- 8.8 Operational security: firewalls and IDS

11/25/2013

Network Security (SSL) 8-19

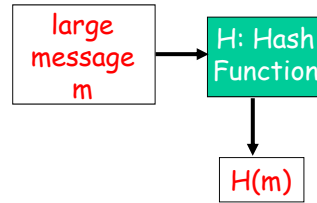
Message Integrity

- Allows communicating parties to verify that received messages are *authentic*.
 - Content of message has not been altered
 - Source of message is who/what you think it is
 - Message has not been replayed
 - Sequence of messages is maintained
- Use of message digests and “nonces”

11/25/2013

Network Security (SSL) 8-20

Message Digests



- Function $H()$ that takes as input an *arbitrary* length message and outputs a *fixed-length* string
 - $H()$ is a many-to-1 function
 - $H()$ is often called a "hash function"

- Desirable properties of **crypto hash** function:
 - Easy to calculate
 - **Seemingly random output**
 - **Irreversible**: Can't determine m from $H(m)$
 - **Collision resistant**: computationally difficult to produce m and m' such that

$$H(m) = H(m')$$

11/25/2013

Network Security (SSL) 8-21

Internet checksum: poor crypto hash function

Internet checksum has some properties of hash function:

- ✓ produces fixed length digest (16-bit sum) of message
- ✓ is many-to-one

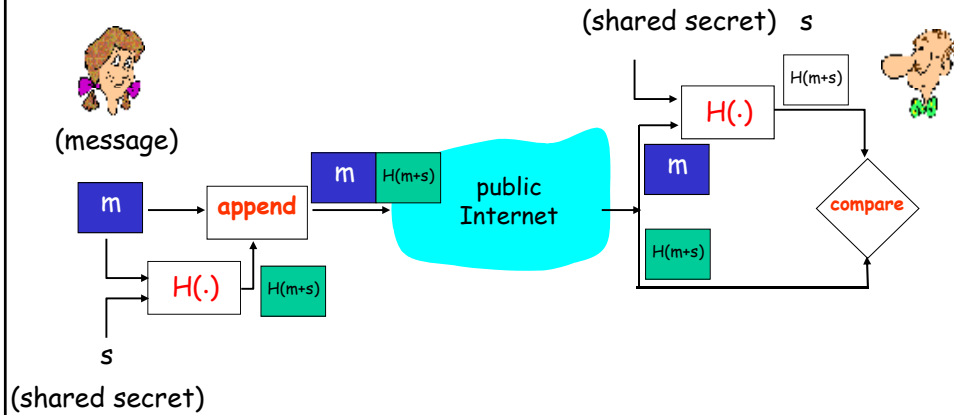
But for a message with given hash value, it is easy to find another message with same hash value. Simple checksum example:

<u>message</u>	<u>ASCII format</u>		<u>message</u>	<u>ASCII format</u>
I O U 1	49 4F 55 31		I O U 9	49 4F 55 39
0 0 . 9	30 30 2E 39		0 0 . 1	30 30 2E 31
9 B O B	39 42 4F 42		9 B O B	39 42 4F 42
	B2 C1 D2 AC	different messages but identical checksums!		B2 C1 D2 AC

11/25/2013

Network Security (SSL) 8-22

Message Authentication Code (MAC)



11/25/2013

Network Security (SSL) 8-23

Crypto hash functions in practice

- **MD5 hash function widely used (RFC 1321)**
 - computes 128-bit message digest in 4-step process.
 - arbitrary 128-bit string, x ,
 - appears difficult to construct msg m whose MD5 hash is x
 - attacks on MD5 (2005) -> may not be collision resistant
- **SHA-1 is also used**
 - US standard [NIST, FIPS PUB 180-1]
 - 160-bit message digest
 - more robust algorithms such as SHA-224, SHA-256, SHA-384 and SHA-512

11/25/2013

Network Security (SSL) 8-24

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.3 Message integrity & **end point authentication**

8.4 Securing e-mail

8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and IDS

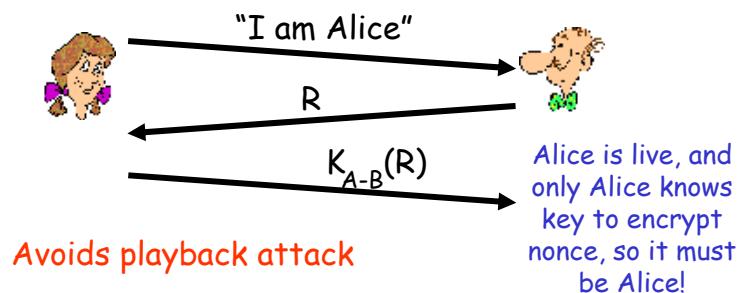
11/25/2013

Network Security (SSL) 8-25

Authentication: challenge-response

Nonce: number (R) used only *once in a lifetime*

To prove that Alice is "live", Bob sends Alice nonce, R.
Alice must return R, encrypted with **shared secret key**



11/25/2013

Network Security (SSL) 8-26

Digital Signature - objectives

- sender (Bob) digitally signs document with his own **private** key, establishing he is document owner/creator - like **hand-written signatures**
- **verifiable, nonforgeable**: recipient (Alice) can prove to someone that Bob, and no one else (including Alice), must have signed document

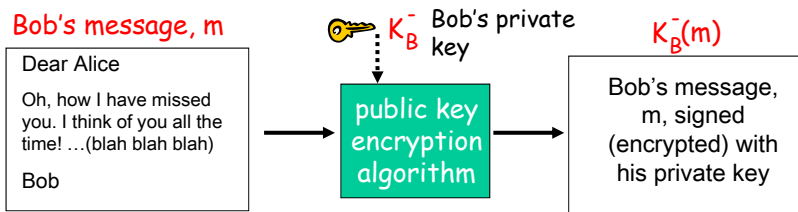
11/25/2013

Network Security (SSL) 8-27

Digital Signature - simple approach

simple digital signature for message m :

- Bob "signs" m by encrypting with his private key K_B^- , creating "signed" message, $K_B^-(m)$



11/25/2013

Network Security (SSL) 8-28

Digital Signature (more)

- suppose Alice receives msg m , digital signature $K_B^-(m)$
- Alice verifies m by using Bob's public key K_B^+ to check $K_B^+(K_B^-(m)) = m$.
- if verified, whoever signed m must have used Bob's private key.

Alice thus verifies that:

- ✓ Bob signed m .
- ✓ No one else signed m .
- ✓ Bob signed m and not m' .

non-repudiation:

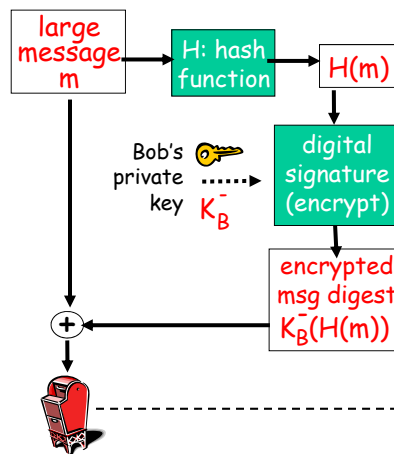
- ✓ Alice (or Bob's wife) can take m , and signature $K_B^-(m)$ to court and prove that Bob signed m or he let someone else use his private key.

11/25/2013

Network Security (SSL) 8-29

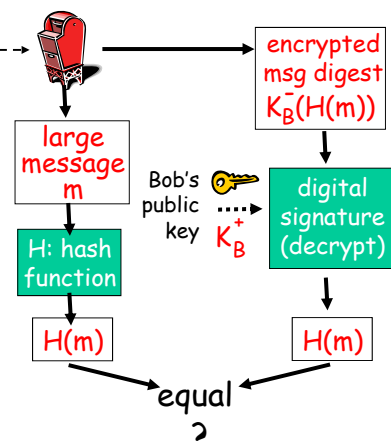
Digital signature - practical approach

Bob sends digitally signed message:



11/25/2013

Alice verifies signature and integrity of digitally signed message:



Network Security (SSL) 8-30

Public Key Certification

public key problem:

- When Alice obtains Bob's public key (from web site, e-mail, etc.), how does she *know* it is Bob's public key, not Trudy's?

solution:

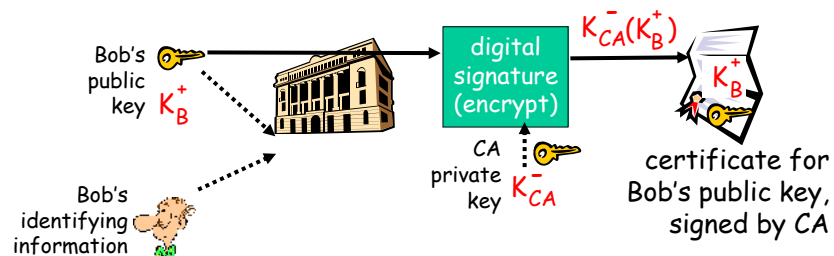
- trusted certification authority (CA)

11/25/2013

Network Security (SSL) 8-31

Certification Authorities

- **Certification Authority (CA):** binds a public key to a particular entity, Bob.
 - Bob provides "proof of identity" to CA.
 - CA creates certificate binding Bob to its public key.
 - certificate containing Bob's public key digitally signed by CA: CA says "This is Bob's public key."

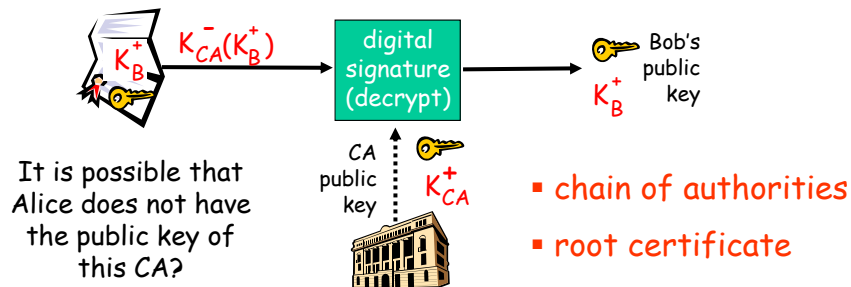


11/25/2013

Network Security (SSL) 8-32

Certification Authorities

- when Alice wants Bob's public key:
 - gets Bob's certificate (from Bob or elsewhere).
 - apply CA's public key to Bob's certificate, get Bob's public key



11/25/2013

Network Security (SSL) 8-33

Chapter 8 roadmap

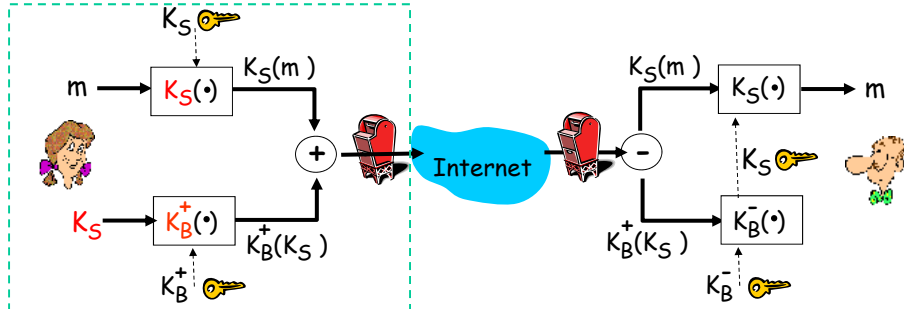
- 8.1 What is network security?
- 8.2 Principles of cryptography
- 8.3 Message integrity & end point authentication
- 8.4 Securing e-mail
- 8.5 Securing TCP connections: SSL
- 8.6 Network layer security: IPsec
- 8.7 Securing wireless LANs
- 8.8 Operational security: firewalls and IDS

11/25/2013

Network Security (SSL) 8-34

Confidential email - sender

- Alice wants to send confidential e-mail, m , to Bob.



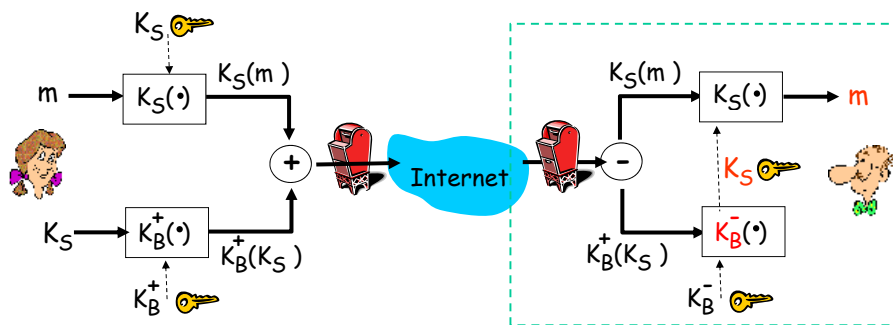
Alice:

- generates random *symmetric* private key, K_S .
- encrypts message with K_S (for efficiency)
- also encrypts K_S with Bob's public key, K_B^+
- sends both $K_S(m)$ and $K_B^+(K_S)$ to Bob.

11/25/2013

Network Security (SSL) 8-35

Confidential email - receiver



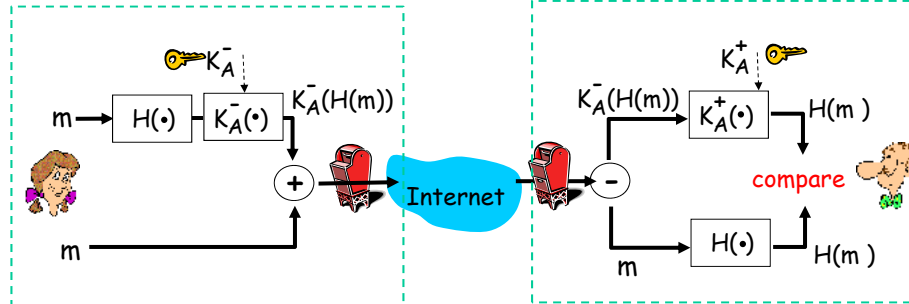
Bob:

- uses his *private* key to decrypt and recover K_S
- uses K_S to decrypt $K_S(m)$ to recover m

11/25/2013

Network Security (SSL) 8-36

Sender authentication and message integrity



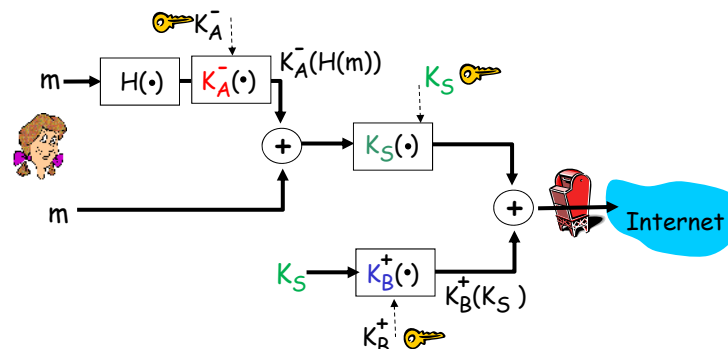
- Alice digitally signs message.
- sends both message (in the clear) and digital signature to Bob.

11/25/2013

Network Security (SSL) 8-37

Secure e-mail

- Alice wants to provide confidentiality, sender authentication, message integrity.



Alice uses three keys: her private key, newly created symmetric key, Bob's public key,

11/25/2013

Network Security (SSL) 8-38

Pretty good privacy (PGP)

- ❑ Internet e-mail encryption de-facto standard. It uses
 - *symmetric key crypto*,
 - *public key crypto*,
 - *hash function*, and
 - *digital signature*
- ❑ provides *confidentiality*, *sender authentication*, *integrity*.
- ❑ Phil Zimmerman (1991)
- ❑ Web of trust model - anyone can sign public key certificates

A PGP signed message:

```
---BEGIN PGP SIGNED MESSAGE--  
-  
Hash: SHA1  
  
Bob:My husband is out of town  
    tonight.Passionately  
    yours, Alice  
  
---BEGIN PGP SIGNATURE---  
Version: PGP 5.0  
Charset: noconv  
yhHJRhhGJGhg/12EpJ+lo8gE4vB3  
mqJhFEvZP9t6n7G6m5Gw2  
---END PGP SIGNATURE---
```

11/25/2013

Network Security (SSL) 8-39

Chapter 8 roadmap

- 8.1 What is network security?
- 8.2 Principles of cryptography
- 8.3 Message integrity & end point authentication
- 8.4 Securing e-mail
- 8.5 **Securing TCP connections: SSL**
- 8.6 Network layer security: IPsec
- 8.7 Securing wireless LANs
- 8.8 Operational security: firewalls and IDS

11/25/2013

Network Security (SSL) 8-40

SSL: Secure Sockets Layer

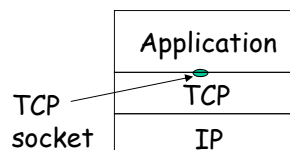
- ❑ widely deployed security protocol
 - supported by almost all browsers, web servers for e-commerce transactions, email, etc.
 - https
 - billions \$/year over SSL
- ❑ origins
 - basic ideas and proof-of-concept version [Woo et al., USENIX June 1994]
 - first deployed version designed and implemented by Netscape

11/25/2013

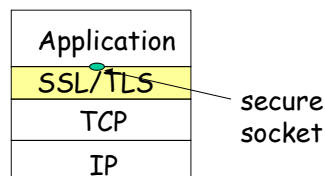
Network Security (SSL) 8-41

Secure sockets layer (more)

- ❑ provides transport layer security to TCP-based apps
 - bottom of application layer using sockets
 - it, in turn, provides **secure sockets**
- ❑ variants: SSL 3.0, TLS 1.0-1.2 (transport layer security)
- ❑ security services:
 - server authentication, confidentiality, integrity, client authentication (optional)



TCP API



TCP enhanced with SSL (in app layer)

11/25/2013

Network Security (SSL) 8-42

SSL: basic protocol steps

- *handshake*: Alice and Bob authenticate each other and exchange a shared secret
- *key derivation*: Alice and Bob use shared secret to derive a set of keys
- *data transfer*: data to be transferred is broken up into series of records
- *connection closure*: special messages to securely close connection

11/25/2013

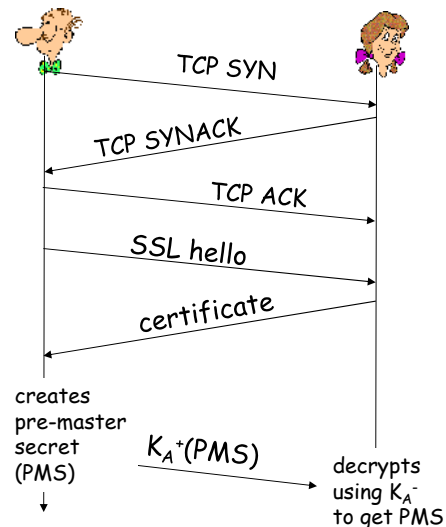
Network Security (SSL)

8-43

SSL: handshake

- Bob (client) establishes TCP connection to Alice (server)
- Bob authenticates Alice from her certificate signed by CA
- Bob creates, encrypts (using Alice's public key) a **pre-master secret** and sends it to Alice

(nonces not shown in messages)



11/25/2013

Network Security (SSL) 8-44

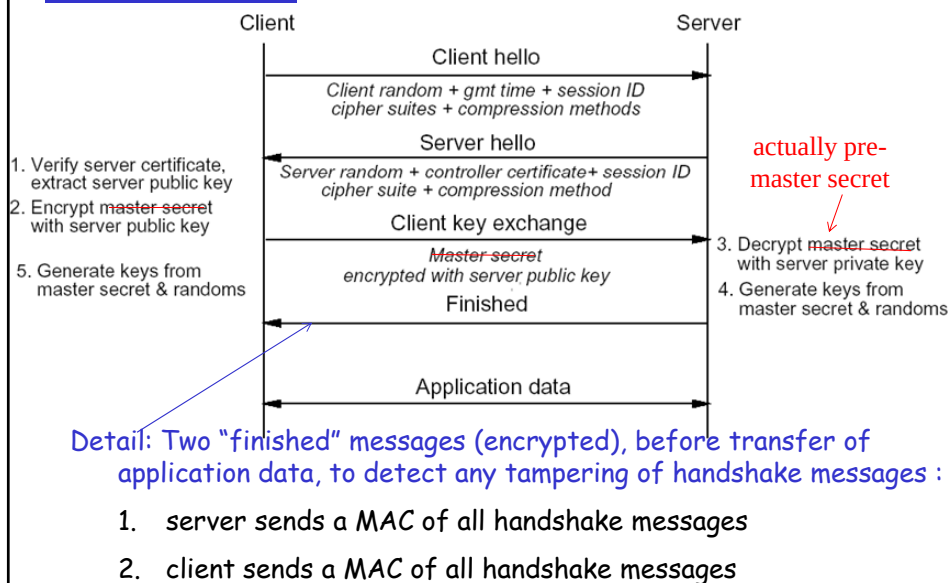
SSL: key derivations

- ❑ Master secret generated from pre-master secret, server and client nonces
- ❑ Alice, Bob use shared master secret to generate 4 symmetric keys:
 - E_B : Bob→Alice data encryption key
 - E_A : Alice→Bob data encryption key
 - M_B : Bob→Alice MAC key
 - M_A : Alice→Bob MAC key
- ❑ encryption and MAC algorithms negotiable between Bob, Alice

11/25/2013

Network Security (SSL) 8-45

More details



11/25/2013

Network Security (SSL) 8-46

SSL record protocol

8 bit 8 bit 8 bit			16 bit	16384 byte
Type	Minor version	Major version	Record Length	Record Data

- ❑ Record protocol provides transport service to handshake protocol, application data, and other protocols (e.g., Alert protocol for ending session)
 - Type is for demultiplexing
- ❑ Application data undergo
 - Fragmentation into blocks
 - Compression (optionally)
 - Computing MAC (Message Authentication Code)
 - using entire record + MAC key + value of sequence counter
 - Encryption applied to data and MAC
 - Type, Version, and Length fields are left in the clear

11/25/2013

Network Security (SSL) 8-47

Chapter 8 roadmap

- 8.1 What is network security?
- 8.2 Principles of cryptography
- 8.3 Message integrity & end point authentication
- 8.4 Securing e-mail
- 8.5 Securing TCP connections: SSL
- 8.6 Network layer security: IPsec
- 8.7 Securing wireless LANs
- 8.8 Operational security: firewalls and IDS

11/25/2013

Network Security (SSL) 8-48

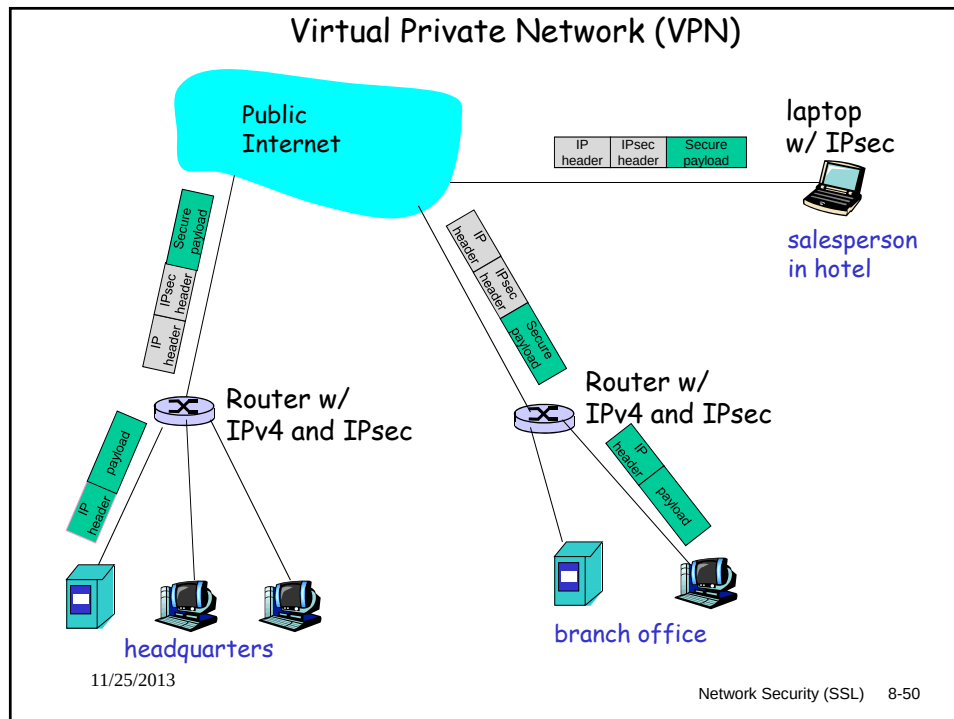
What is confidentiality at the network-layer?

Between two network entities:

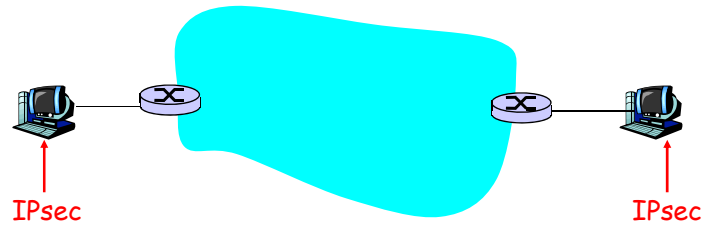
- ❑ Sending entity encrypts the payloads of datagrams. Payload could be:
 - TCP segment, UDP segment, ICMP message, OSPF message, and so on.
- ❑ All data sent from one network entity to the other would be hidden:
 - Web pages, e-mail, P2P file transfers, TCP SYN packets, and so on.
 - That is, "blanket coverage".

11/25/2013

Network Security (SSL) 8-49



IPsec Transport Mode

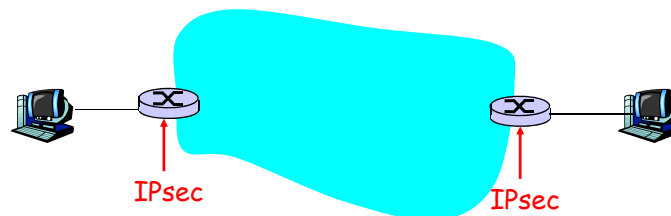


- ❑ IPsec datagrams emitted and received by two end systems.
- ❑ Protects upper-layer protocols of these end systems

11/25/2013

Network Security (SSL) 8-51

IPsec - tunneling mode (1)

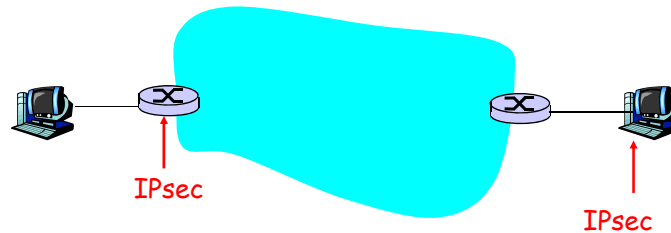


- ❑ End routers are IPsec aware. Hosts need not be.

11/25/2013

Network Security (SSL) 8-52

IPsec - tunneling mode (2)



- Another tunneling mode
 - one host is a laptop or computer at home

11/25/2013

Network Security (SSL) 8-53

Two protocols

- Authentication Header (AH) protocol
 - provides source authentication & data integrity (including replay detection) but *not* confidentiality
 - 51 in protocol field of IP header
- Encapsulation Security Protocol (ESP)
 - provides source authentication, data integrity (including replay detection), *and confidentiality*
 - 50 in protocol field of IP header

11/25/2013

Network Security (SSL) 8-54

Four combinations are possible

Transport mode with AH	Transport mode with ESP
Tunnel mode with AH	Tunnel mode with ESP

Most common and
most important

11/25/2013

Network Security (SSL) 8-55

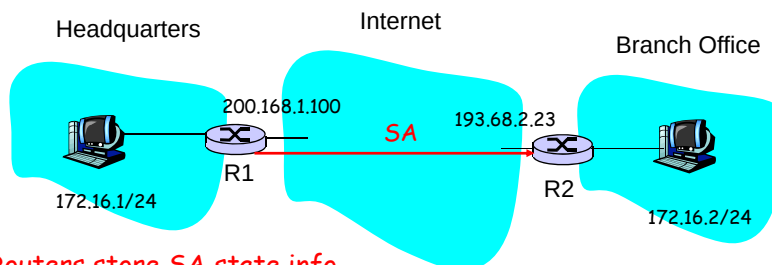
Security association

- ❑ Before sending data, a logical connection is established from sending entity to receiving entity, called **security association (SA)**
 - SAs are **unidirectional**
- ❑ Both sending and receiving entities maintain **state information** for the SAs
 - IP is **connectionless** but IPsec is **connection-oriented**
- ❑ How many SAs maintained at headquarters to communicate with a branch office and n traveling salespeople?

11/25/2013

Network Security (SSL) 8-56

Example SA from R1 to R2



Routers store SA state info

- ❑ 32-bit identifier for SA: *Security Parameter Index (SPI)*
- ❑ the origin interface of the SA (200.168.1.100)
- ❑ destination interface of the SA (193.68.2.23)
- ❑ type of encryption to be used (for example, 3DES with CBC)
- ❑ encryption key
- ❑ type of integrity check (for example, HMAC with MD5)
- ❑ authentication key

11/25/2013

Network Security (SSL) 8-57

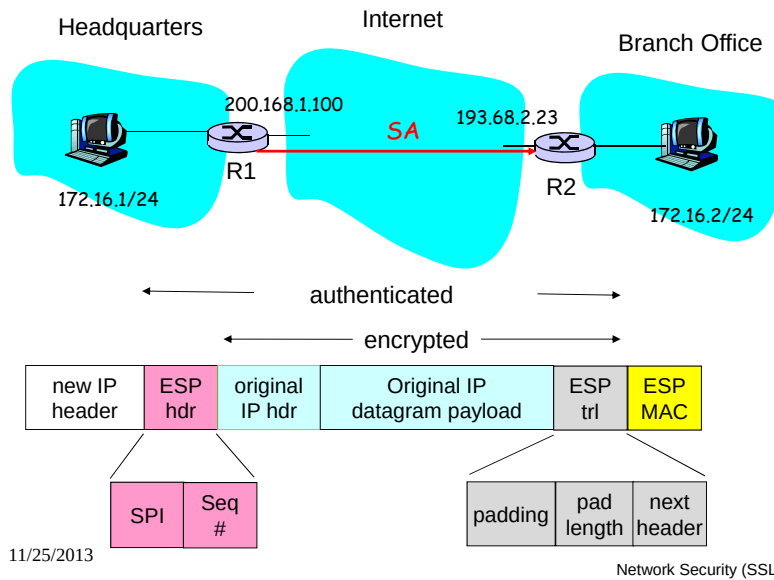
Security Association Database (SAD)

- ❑ Endpoint holds state info of its SAs in a SAD
- ❑ With n salespeople and one branch office, $2 + 2n$ SAs in router R1's SAD
- ❑ When sending IPsec datagram, R1 accesses SAD to determine how to process datagram.
- ❑ When IPsec datagram arrives at R2, R2 retrieves SPI, indexes its SAD with SPI, and processes datagram accordingly.

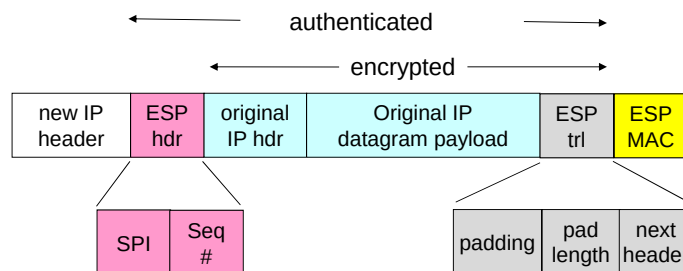
11/25/2013

Network Security (SSL) 8-58

IPsec datagram - tunnel mode, ESP



Inside the packet:



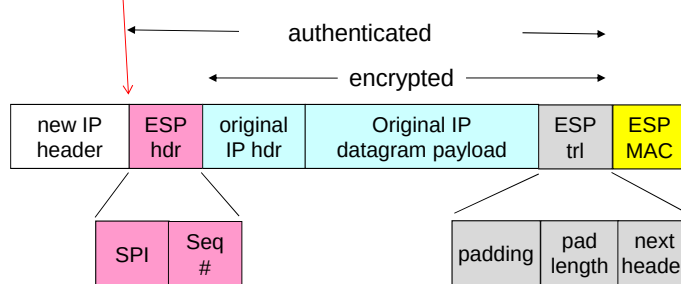
- ❑ ESP header:
 - SPI, so receiving entity knows what to do
 - Sequence number, to thwart replay attacks
- ❑ ESP trailer: Padding for block ciphers
 - next header identifies upper-layer protocol in payload field
- ❑ Use shared keys to encrypt and create ESP MAC field

What about
NAT
traversal?

11/25/2013

Network Security (SSL) 8-60

When a tunnel endpoint is behind a NAT



- ❑ NAT needs a port number - original port number is encrypted
- ❑ **Solution: wrap IPsec packet with UDP header**
 - ❑ port number in UDP header identifies IPsec

11/25/2013

Network Security (SSL) 8-61

Summary: IPsec services

- ❑ Suppose Trudy sits somewhere between R1 and R2. She doesn't know the keys.
 - Will Trudy be able to see contents of original datagram? **How about source, destination IP addresses, transport protocol, application port?**
 - Flip bits without detection?
 - **Masquerade as R1 using R1's IP address?**
 - Replay a datagram?

11/25/2013

Network Security (SSL) 8-62

Authentication methods

- ❑ PSK (pre-shared key)
- ❑ PKI (public key infrastructure)
 - public/private keys and certificates

11/25/2013

Network Security (SSL) 8-63

IKE: Key Management in IPsec

- ❑ In previous examples, we manually established IPsec SAs in IPsec endpoints:

Example SA

SPI: 12345
Source IP: 200.168.1.100
Dest IP: 193.68.2.23
Protocol: ESP
Encryption algorithm: 3DES-cbc
HMAC algorithm: MD5
Encryption key: 0x7aeaca...
HMAC key: 0xc0291f...

- ❑ Such manual keying is impractical for large VPN
- ❑ Instead, use *IPsec IKE (Internet Key Exchange)*

11/25/2013

Network Security (SSL) 8-64

Two Phases of IKE

- ❑ Phase I: Establish *bi-directional* IKE security association (SA)
 - IKE SA is different from IPsec SA
 - Each endpoint has private and public keys
 - Endpoints use Diffie-Hellman algorithm to establish a *shared secret*
- (Mutual authentication is not done in phase I)

11/25/2013

Network Security (SSL) 8-65

Diffie-Hellman algorithm

- ❑ Publicly known large prime number p and a number g that is a primitive root mod p
- ❑ Alice and Bob independently choose *private keys*, α and β , respectively and exchange their public keys
 - *public key of Alice* is $g^\alpha \bmod p$
 - *public key of Bob* is $g^\beta \bmod p$
- ❑ Shared secret known only to Alice and Bob is
$$g^{\alpha\beta} \bmod p = (g^\beta \bmod p)^\alpha \bmod p = (g^\alpha \bmod p)^\beta \bmod p$$
- ❑ Note that in phase I, public keys of Alice and Bob have not been authenticated

11/25/2013

Network Security (SSL) 8-66

Two Phases of IKE (cont.)

- Phase I: Establish bi-directional IKE SA
- Phase II: The endpoints *reveal their identities* to each other and mutually authenticate using public key certificates
 - Identities are *not revealed to passive sniffers* since messages are sent over the encrypted IKE SA channel between the endpoints
 - The two sides create SA for each direction, negotiate encryption and authentication algorithms, establish session keys for encryption and authentication

11/25/2013

Network Security (SSL) 8-67

Chapter 8 roadmap

- 8.1 What is network security?
- 8.2 Principles of cryptography
- 8.3 Message integrity & end point authentication
- 8.4 Securing e-mail
- 8.5 Securing TCP connections: SSL
- 8.6 Network layer security: IPsec
- 8.7 Securing wireless LANs
- 8.8 Operational security: firewalls and IDS

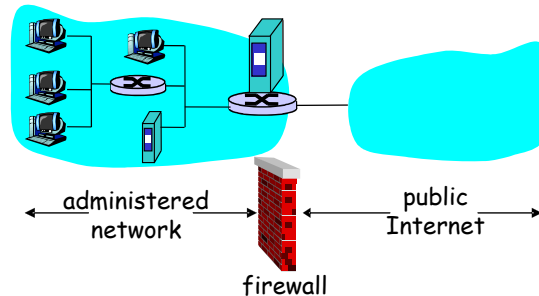
11/25/2013

Network Security (SSL) 8-68

Firewalls

firewall

isolates organization's internal net from public Internet, allowing some packets to pass, blocking others (i.e., **access control**)



11/25/2013

Network Security (SSL) 8-69

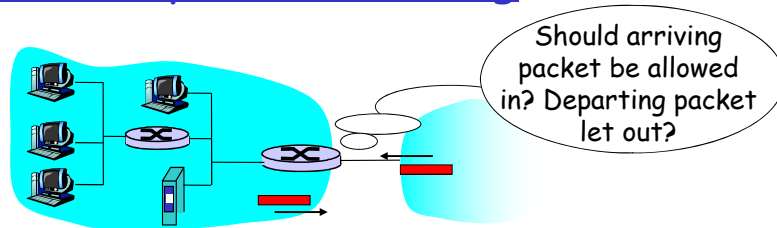
Three types of firewalls

- ☐ stateless packet filters
- ☐ stateful packet filters
- ☐ application gateways (in servers rather than routers)

11/25/2013

Network Security (SSL) 8-70

Stateless packet filtering



- ❑ internal network connected to Internet via **router firewall**
- ❑ router **filters packet by packet**, decision to forward/drop packet based on:
 - source IP address, destination IP address
 - TCP/UDP source and destination port numbers
 - ICMP message type
 - TCP SYN and ACK bits

11/25/2013

Network Security (SSL) 8-71

Stateless packet filtering: example

- ❑ **example 1: block incoming and outgoing datagrams with IP protocol field = 17 and with either source or dest port = 23.**
 - all incoming, outgoing UDP flows and telnet connections are blocked.
- ❑ **example 2: Block inbound TCP segments with ACK=0.**
 - prevents external clients from making TCP connections with internal hosts, but allows internal clients to connect to outside.

11/25/2013

Network Security (SSL) 8-72

Access Control Lists

- ❑ **ACL:** list of rules, apply to every incoming packet
- ❑ **first match** determines action to take on packet

action	source address	dest address	protocol	source port	dest port	flag bit
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80	any
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	---
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	----
deny	all	all	all	all	all	all

11/25/2013

Network Security (SSL) 8-73

Stateful packet filtering

- ❑ **stateless packet filter: memoryless**
 - May admit packets that "make no sense," e.g., source port = 80, ACK bit set, even though no TCP connection has been established, e.g.,

action	source address	dest address	protocol	source port	dest port	flag bit
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK

- ❑ **stateful packet filter:** track status of every TCP connection
 - track connection setup (SYN), teardown (FIN): can determine whether incoming, outgoing packets "make sense"
 - timeout inactive connections at firewall: no longer admit their packets

11/25/2013

Network Security (SSL) 8-74

Stateful packet filtering

- ACL augmented to indicate need to check **connection state table** before admitting packet

action	source address	dest address	proto	source port	dest port	flag bit	check connexion
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80	any	
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK	×
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	---	
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	----	×
deny	all	all	all	all	all	all	

11/25/2013

Network Security (SSL) 8-75

Network Security (summary)

Basic techniques.....

- cryptography (symmetric and public)
- message integrity
- end-point authentication

.... used in many different security scenarios

- secure email
- secure transport (SSL)
- IP sec

Operational Security: firewalls and ...

11/25/2013

Network Security (SSL) 8-76